Confluent Cloud Python Dynamic or Static Protobuf Example

A hands-on Python demonstration of the **Confluent Cloud Protobuf Schema Serializer & Deserializer**, covering every major concept from the official Confluent Protobuf SerDes documentation.

The project talks to a Confluent Cloud Schema Registry over the SR REST API and, when run in `full` mode, produces and consumes messages on a Kafka cluster via `confluent-kafka`. It supports two Protobuf modes:

- **Dynamic (default)** — schemas are defined as Python dataclasses, compiled at runtime into `google.protobuf` `FileDescriptorProto` objects, and serialized to **Protobuf binary** using `google.protobuf.message_factory`. No `protoc` compiler or generated stubs are required.
- **Static (`--use-protoc`)** — schemas are pre-compiled from `.proto` files in the `schemas/` directory into `_pb2.py` stubs via `protoc`, then wrapped by `CompiledProtoMessage` for better performance and compile-time type safety.

Both modes satisfy the `ProtoSchema` protocol and are interchangeable in the SerDes layer.

Table of Contents

- 1.0 Anatomy & Setup
  - 1.1 Project layout
  - 1.2 Architecture Overview
  - 1.3 Requirements
    - 1.3.1 Install uv
      - 1.3.1.1 Special mention
  - 1.4 Setup
  - 1.5 Dependencies
  - 1.6 Configuration
  - 1.7 Confluent Cloud prerequisites
  - 1.8 Core classes
    - 1.8.1 `SchemaRegistryClient` (`schema_registry_client.py`)
    - 1.8.2 `ProtoSchema` (`proto_schema.py`)
    - 1.8.3 `ProtoMessage` / `ProtoField` (`dynamic_protobuf_helpers.py`)
    - 1.8.4 `CompiledProtoMessage` / `compile_protos` / `load_compiled_message` (`compiled_protobuf_helpers.py`)
    - 1.8.5 `KafkaProtobufSerializer` (`kafka_protobuf_serdes.py`)
    - 1.8.6 `KafkaProtobufDeserializer` (`kafka_protobuf_serdes.py`)
    - 1.8.7 `FieldEncryptor` / `get_encrypted_fields()` (`field_encryption.py`)
    - 1.8.8 `kafka_helpers.py`
    - 1.8.9 `utilities.py`
    - 1.8.10 `demos.py`
  - 1.9 Logging
  - 1.10 Wire format

- 1.10.1 Why a message index?
- 1.10.2 What the code does
- 2.0 Running the demos
  - 2.1 Demos
    - 2.1.1 Demo 1 — Basic Serializer & Deserializer (`--demo basic`)
    - 2.1.2 Demo 2 — Reference-Deletion Protection (`--demo delete-protection`)
    - 2.1.3 Demo 3 — Schema Evolution (`--demo evolution`)
    - 2.1.4 Demo 4 — Multiple Event Types / `oneOf` (`--demo oneof`)
    - 2.1.5 Demo 5 — Null-Value Handling (`--demo null-handling`)
    - 2.1.6 Demo 6 — Compatibility Rules (`--demo compatibility`)
    - 2.1.7 Demo 7 — Schema Types (`--demo types`)
    - 2.1.8 Demo 8 — Subject Name Strategies (`--demo strategies`)
    - 2.1.9 Demo 9 — Client-Side Field Level Encryption (`--demo csfle`)
    - 2.1.10 Demo 10 — Manual Schema Registration (`--demo no-auto-register`)
- 3.0 Cleanup
- 4.0 Notes
- 5.0 Resources

1.0 Anatomy & Setup

1.1 Project layout

```
cc-python-dynamic_static-protobuf-example/
├── src
│   ├── constants.py                # DEFAULT_TOOL_LOG_FILE,
DEFAULT_TOOL_LOG_FORMAT
│   ├── utilities.py                # setup_logging(), get_config(),
parse_args() — logging, env config, CLI
│   ├── schema_registry_client.py   # SchemaRegistryClient — SR REST API
wrapper + wire format
│   ├── proto_schema.py             # ProtoSchema Protocol — common
interface for dynamic & compiled schemas
│   ├── dynamic_protobuf_helpers.py  # ProtoMessage & ProtoField — dynamic
proto3 schema builders
│   ├── compiled_protobuf_helpers.py # CompiledProtoMessage,
compile_protos(), load_compiled_message() — protoc stubs
│   ├── kafka_protobuf_serdes.py     # KafkaProtobufSerializer &
KafkaProtobufDeserializer
│   ├── kafka_helpers.py            # ensure_topics(), kafka_produce(),
kafka_consume_one()
│   ├── field_encryption.py         # FieldEncryptor &
get_encrypted_fields() — AES-256-GCM CSFLE
│   ├── demos.py                    # All ten demo functions (demo_basic
… demo_no_auto_register)
│   ├── main.py                     # Thin entry point — wires config, SR
client, and demo dispatch
│   └── generated_pb2/              # Auto-generated protoc stubs
(gitignored, created by --use-protoc)
├── schemas                         # Proto3 schema definitions
```

```
(gitignored, used by --use-protoc)
│   ├── MyRecord.proto              # Basic schema with import
│   ├── other.proto                  # Referenced schema (OtherRecord)
│   ├── AllTypes.proto               # OneOf wrapper (Customer + Product
+ Order)
│   ├── Customer.proto               # Customer message
│   ├── Product.proto                # Product message
│   ├── Order.proto                  # Order message
│   ├── Payment.proto                # Payment message (subject name
strategies)
│   ├── SensitiveRecord.proto        # CSFLE example with PII fields
│   ├── ExampleMessage.proto         # General-purpose example
│   ├── Invoice.proto                # Manual registration example (no-
auto-register demo)
│   └── evolution
│       ├── MyRecord_v1.proto        # Schema evolution v1 (id, amount)
│       └── MyRecord_v2.proto        # Schema evolution v2 (+
customer_id)
├── run-demo.sh                     # Shell script — authenticates via
AWS SSO and runs all demos in `full` mode
├── pyproject.toml                  # Project metadata, dependencies,
logging
├── uv.lock                         # Pinned dependency lockfile — commit
this
├── .env                            # Credentials — NOT COMMITTED, loaded
automatically by python-dotenv at startup
├── .gitignore                      # Ignore .env, .venv/, logs,
__pycache__/, schemas/, generated_pb2/, etc.
├── CHANGELOG.md                    # Changelog in Markdown format
├── CHANGELOG.pdf                   # Changelog in PDF format
├── KNOWNISSUES.md                  # Known issues in Markdown format
├── KNOWNISSUES.pdf                 # Known issues in PDF format
├── LICENSE.md                      # License in Markdown format
├── LICENSE.pdf                     # License in PDF format
└── README.md                       # This file — project overview, setup
instructions, and documentation of all core concepts and classes
```

1.2 Architecture Overview

```
flowchart TB

    %% ── Bootstrap ──────────────────────────────────────────────────
    subgraph Boot["Startup"]
        direction LR
        ENV[".env file"]
        PYPROJECT["pyproject.toml\n[tool.logging]"]

CONST["constants.py\nDEFAULT_TOOL_LOG_FILE\nDEFAULT_TOOL_LOG_FORMAT"]
        UTIL["utilities.py\nsetup_logging()\ntomllib.load()"]
        LOG["logging.config\n.dictConfig()"]
        DOTENV["load_dotenv()"]
```

```
        CONST --> UTIL
        PYPROJECT --> UTIL
        UTIL --> LOG
        ENV --> DOTENV
    end

    %% ── CLI ──────────────────────────────────────────────────────────
    subgraph CLI["CLI  (main.py · utilities.py)"]
        direction LR
        ARGS["parse_args()\n--mode  --demo  --run-id\n--save-schemas"]
        CFG["get_config()\nreads os.environ"]
        ARGS --> CFG
    end

    Boot --> CLI

    %% ── ProtoSchema Protocol ─────────────────────────────────────────
    subgraph PROTO_IF["ProtoSchema Protocol  (proto_schema.py)"]
        direction LR
        PSPROTO["ProtoSchema\nname · 
file_name\nto_schema_string()\nserialize() · 
deserialize()\nsave_schema()"]
    end

    %% ── Dynamic schema helpers 
──────────────────────────────────────────
    subgraph SCHEMA["Dynamic Proto Helpers 
(dynamic_protobuf_helpers.py)"]
        direction TB
        PF["ProtoField\nname · type · number\noptional · repeated"]
        PM["ProtoMessage\nname · package · imports\nfields · oneofs"]
        TSS["to_schema_string()\n→ proto3 text"]
        SER["serialize(data)\nParseDict → SerializeToString\n→ Protobuf 
binary"]
        DES["deserialize(raw)\nFromString → MessageToDict\n→ dict"]
        PM --> TSS
        PM --> SER
        PM --> DES
        PF --> PM
    end

    %% ── Compiled schema helpers 
────────────────────────────────────────────
    subgraph COMPILED["Compiled Proto Helpers 
(compiled_protobuf_helpers.py)"]
        direction TB
        COMP["compile_protos()\nprotoc → _pb2.py stubs"]
        LOAD["load_compiled_message()\nimport _pb2 module"]
        CPM["CompiledProtoMessage\nname · file_name\nserialize() · 
deserialize()"]
        COMP --> LOAD
        LOAD --> CPM
    end
```

```
    PM -.->|"satisfies"| PSPROTO
    CPM -.->|"satisfies"| PSPROTO

    %% ── SR client
────────────────────────────────────────────────────────
    subgraph SRC["SchemaRegistryClient  (schema_registry_client.py)"]
        direction TB
        SESS["requests.Session\nBasic auth (SR key/secret)\nContent-Type:
vnd.schemaregistry.v1+json"]
        CACHE["_cache: dict[int, dict]\nschema_id → schema_obj"]

        subgraph SR_READ["Read"]
            GET_TYPES["GET /schemas/types"]
            GET_SUBJ["GET /subjects"]
            GET_VERS["GET /subjects/{s}/versions"]
            GET_VER["GET /subjects/{s}/versions/{v}"]
            GET_ID["GET /schemas/ids/{id}  ← cached"]
            GET_ID_VERS["GET /schemas/ids/{id}/versions"]
            GET_REFBY["GET /subjects/{s}/versions/{v}/referencedby"]
        end

        subgraph SR_WRITE["Write / Delete"]
            POST_REG["POST /subjects/{s}/versions\nregister schema →
schema_id"]
            DEL_VER["DELETE /subjects/{s}/versions/{v}"]
            DEL_SUBJ["DELETE /subjects/{s}  soft delete"]
            DEL_PERM["DELETE /subjects/{s}?permanent=true"]
        end

        subgraph SR_CSFLE["Catalog / DEK Registry (CSFLE)"]
            POST_TAG["POST /catalog/v1/types/tagdefs\ncreate_tag()
idempotent"]
            POST_KEK["POST /dek-registry/v1/keks\ncreate_kek()"]
            GET_KEK["GET /dek-registry/v1/keks/{name}\nget_kek()"]
            POST_DEK["POST /dek-
registry/v1/keks/{name}/deks\ncreate_dek()"]
            GET_DEK["GET /dek-
registry/v1/keks/{name}/deks/{subject}\nget_dek()"]
        end

        subgraph SR_COMPAT["Compatibility"]
            POST_COMPAT["POST /compatibility/subjects/{s}/versions/{v}\n→
is_compatible"]
            GET_CFG["GET /config[/{s}]\n→ compatibilityLevel"]
            PUT_CFG["PUT /config[/{s}]\nset level e.g.
BACKWARD_TRANSITIVE"]
        end

        subgraph WIRE["Wire Format  (local)"]
            ENC["encode(schema_id, payload)\nstruct.pack('>bI', 0x00, id)
+ payload"]
            DEC["decode_header(data)\nvalidate magic
0x00\nstruct.unpack('>I', data[1:5])"]
```

```
        end

        SESS --> SR_READ
        SESS --> SR_WRITE
        SESS --> SR_COMPAT
        SESS --> SR_CSFLE
        GET_ID --> CACHE
    end

    %% ── SerDes
────────────────────────────────────────────────────────────────
    subgraph SERDES["SerDes  (kafka_protobuf_serdes.py)"]
        direction LR
        subgraph KSER["KafkaProtobufSerializer"]
            STRAT["SubjectNameStrategy\nTopicName · 
RecordName\nTopicRecordName"]
            REFSTRAT["ReferenceSubjectNameStrategy\nDefault · Qualified"]
            AUTO["auto_register=True\n→ sr.register()\nor 
sr.get_version()"]
            KSER_OUT["sr.encode(schema_id, payload)\n→ wire bytes"]
            STRAT --> AUTO
            AUTO --> KSER_OUT
        end
        subgraph KDES["KafkaProtobufDeserializer"]
            KDES_IN["sr.decode_header(raw)\n→ schema_id + payload"]
            WARM["sr.get_schema_by_id()\nwarm cache"]
            SPEC["specific_type set?\n→ specific_type.deserialize()\nelse 
_schema_id_to_message lookup"]
            KDES_IN --> WARM
            WARM --> SPEC
        end
    end

    %% ── Kafka helpers
──────────────────────────────────────────────────
    subgraph KAFKA_H["Kafka Helpers  (kafka_helpers.py · --mode full 
only)"]
        direction TB

BASE["_base_kafka_config()\nbootstrap.servers\nsecurity.protocol=SASL_SSL\
nsasl.mechanisms=PLAIN"]
        ADMIN_H["ensure_topics()\nAdminClient → 
list_topics()\nNewTopic(rf=3, partitions=6)\ncreate_topics()  idempotent"]
        PROD_H["kafka_produce()\nProducer → produce() + flush()"]
        CONS_H["kafka_consume_one()\nConsumer → subscribe()\npoll() loop + 
commit()"]
        BASE --> ADMIN_H
        BASE --> PROD_H
        BASE --> CONS_H
    end

    %% ── CSFLE ──────────────────────────────────────────────────────────
    subgraph CSFLE_BOX["CSFLE  (field_encryption.py)"]
        direction TB
```

```
        FE["FieldEncryptor\nAES-256-GCM\nAWS KMS + DEK Registry"]
        GEF["get_encrypted_fields()\nmetadata tags → field list"]
        GEF --> FE
    end

    %% ── Demos
────────────────────────────────────────────────────────────────
    subgraph DEMOS["Demo Sections  (demos.py)"]
        direction LR
        D1["Demo 1 · Basic\ntestproto-{run_id}"]
        D2["Demo 2 · Delete Protection\nreferencedby → correct order"]
        D3["Demo 3 · Schema Evolution\nBACKWARD_TRANSITIVE\ntransactions-
proto-{run_id}"]
        D4["Demo 4 · oneOf\nAllTypes wrapper\nall-events-{run_id}"]
        D5["Demo 5 · Null Handling\noptional fields\nnullables-{run_id}"]
        D6["Demo 6 · Compatibility\n7 modes reference table"]
        D7["Demo 7 · Schema Types\nspecific vs DynamicMessage"]
        D8["Demo 8 · Subject Strategies\npayments-{run_id}"]
        D9["Demo 9 · CSFLE\nfield-level encryption\ncsfle-{run_id}"]
        D10["Demo 10 · Manual Registration\nauto_register=False\ninvoices-
{run_id}"]
    end

    %% ── Confluent Cloud
──────────────────────────────────────────────────
    subgraph CC["Confluent Cloud"]
        CC_SR["Schema Registry\nStream Governance\nEssentials / Advanced"]
        CC_KAFKA["Kafka Cluster\nSASL_SSL · PLAIN\nrf=3"]
    end

    %% ── Wiring
────────────────────────────────────────────────────────────────
    CLI --> SRC
    CLI --> SCHEMA
    CLI --> COMPILED
    CLI --> SERDES
    CLI --> KAFKA_H
    CLI --> DEMOS
    CLI --> CSFLE_BOX

    SCHEMA --> SERDES
    COMPILED --> SERDES
    PROTO_IF --> SERDES
    SRC --> SERDES
    SRC --> DEMOS
    SERDES --> DEMOS
    KAFKA_H --> DEMOS

    CSFLE_BOX --> SERDES
    CSFLE_BOX --> DEMOS

    SRC  -->|"HTTPS REST"| CC_SR
    KAFKA_H -->|"SASL_SSL"| CC_KAFKA
```

```
    %% ── Styles
────────────────────────────────────────────────────────────────────
    classDef cloud    fill:#0a74da,color:#fff,stroke:#0a74da
    classDef core     fill:#1a6b3c,color:#fff,stroke:#1a6b3c
    classDef helper   fill:#6b4226,color:#fff,stroke:#6b4226
    classDef demo     fill:#5a3472,color:#fff,stroke:#5a3472
    classDef wire     fill:#b85c00,color:#fff,stroke:#b85c00
    classDef boot     fill:#2b4a6b,color:#fff,stroke:#2b4a6b

    class CC_SR,CC_KAFKA cloud
    class SRC,SERDES core
    class KAFKA_H,SCHEMA,COMPILED helper
    class DEMOS,D1,D2,D3,D4,D5,D6,D7,D8,D9,D10 demo
    class WIRE,ENC,DEC wire
    class Boot,UTIL,CONST boot
    classDef csfle    fill:#8b1a1a,color:#fff,stroke:#8b1a1a
    class CSFLE_BOX,FE,GEF csfle
    class SR_CSFLE,POST_TAG,POST_KEK,GET_KEK,POST_DEK,GET_DEK csfle
    classDef iface     fill:#4a4a4a,color:#fff,stroke:#4a4a4a
    class PROTO_IF,PSPROTO iface
```

1.3 Requirements

- **Python ≥ 3.13**
- **uv** — package and project manager

1.3.1 Install uv

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# or via Homebrew
brew install uv
```

1.3.1.1 Special mention

You maybe asking yourself why `uv`? Well, `uv` is an incredibly fast Python package installer and dependency resolver, written in **Rust**, and designed to seamlessly replace `pip`, `pipx`, `poetry`, `pyenv`, `twine`, `virtualenv`, and more in your workflows. By prefixing `uv run` to a command, you're ensuring that the command runs in an optimal Python environment.

Now, let's go a little deeper into the magic behind `uv run`:

- When you use it with a file ending in `.py` or an HTTP(S) URL, `uv` treats it as a script and runs it with a Python interpreter. In other words, `uv run file.py` is equivalent to `uv run python file.py`. If you're working with a URL, `uv` even downloads it temporarily to execute it. Any inline dependency

metadata is installed into an isolated, temporary environment—meaning zero leftover mess! When used with `-`, the input will be read from `stdin`, and treated as a Python script.
- If used in a project directory, `uv` will automatically create or update the project environment before running the command.
- Outside of a project, if there's a virtual environment present in your current directory (or any parent directory), `uv` runs the command in that environment. If no environment is found, it uses the interpreter's environment.

So what does this mean when we put `uv run` before `python`? It means `uv` takes care of all the setup—fast and seamless—right in your local Docker container. If you think AI is magic, the work the folks at Astral have done with `uv` is pure wizardry!

Curious to learn more about Astral's `uv`? Check these out:

- Documentation: Learn about `uv`.
- Video: `uv` IS THE FUTURE OF PYTHON PACKING!

1.4 Setup

```
git clone https://github.com/j3-signalroom/cc-python-dynamic_static-protobuf-example
cd cc-python-dynamic_static-protobuf-example

# Create .venv and install exact pinned versions from uv.lock
uv sync
```

`uv sync` reads both `pyproject.toml` and `uv.lock` and installs everything into a local `.venv`. No manual `pip install` is needed.

1.5 Dependencies

Package	Minimum	Purpose
`aws2-wrap`	1.4.0	AWS SSO credential wrapper used by `run-demo.sh`
`boto3`	1.38.0	AWS KMS client for CSFLE DEK unwrapping
`confluent-kafka`	2.13.2	Producer, Consumer, AdminClient (required for `--mode full` only)
`cryptography`	44.0.0	AES-256-GCM encryption for Client-Side Field Level Encryption (CSFLE)
`protobuf`	7.34.0	`google.protobuf` runtime (available for real binary encoding)
`requests`	2.32.5	Schema Registry REST API calls
`python-dotenv`	1.2.2	Auto-loads `.env` via `load_dotenv()` at startup
`dotenv`	0.9.9	dotenv compatibility shim

> `confluent-kafka` is imported inside a `try/except` at startup; if it is absent the app still runs normally in `--mode schema-only`.

1.6 Configuration

Create a `.env` file in the project root (never commit it):

```
# Kafka cluster — only required for --mode full
BOOTSTRAP_SERVERS=pkc-xxxxx.us-east1.gcp.confluent.cloud:9092
KAFKA_API_KEY=your_kafka_api_key
KAFKA_API_SECRET=your_kafka_api_secret

# Schema Registry — always required
SCHEMA_REGISTRY_URL=https://psrc-xxxxx.us-east-2.aws.confluent.cloud
SR_API_KEY=your_sr_api_key
SR_API_SECRET=your_sr_api_secret

# AWS KMS — required for --demo csfle (or --demo all)
AWS_KMS_KEY_ARN=arn:aws:kms:us-east-1:123456789012:key/your-key-id
```

`python-dotenv` loads this automatically at module startup via `load_dotenv()`; no `--env-file` flag is needed.

1.7 Confluent Cloud prerequisites

Resource	Minimum
Schema Registry	Stream Governance **Advanced** (required for DEK Registry / CSFLE)
SR API key	`DeveloperRead` + `DeveloperWrite` on Schema Registry
Kafka cluster	Any type — Basic, Standard, Dedicated, or Enterprise
Kafka API key	`DeveloperRead` + `DeveloperWrite` on the cluster
AWS KMS key	Symmetric encrypt/decrypt key — required for `--demo csfle`
AWS credentials	`boto3`-compatible auth (env vars, `~/.aws/credentials`, IAM role, or AWS SSO — see `run-all-demos.sh`)

1.8 Core classes

1.8.1 `SchemaRegistryClient` (`schema_registry_client.py`)

A `requests.Session`-based REST client covering the full SR API surface used by the demos. Authenticates with HTTP Basic (SR API key/secret). Maintains an in-process `_cache: dict[int, dict]` keyed by schema ID to avoid redundant `GET /schemas/ids/{id}` round-trips. Also includes the `_read_varint()` helper used by `decode_header()` to skip the message-index varint array.

Method	HTTP endpoint
`get_types()`	`GET /schemas/types`
`get_subjects()`	`GET /subjects`
`get_versions(subject)`	`GET /subjects/{s}/versions`
`get_version(subject, version)`	`GET /subjects/{s}/versions/{v}`
`get_schema_by_id(id)`	`GET /schemas/ids/{id}` *(cached)*
`get_versions_for_schema(id)`	`GET /schemas/ids/{id}/versions`
`referenced_by(subject, version)`	`GET /subjects/{s}/versions/{v}/referencedby`
`register(subject, schema, ...)`	`POST /subjects/{s}/versions`
`create_tag(tag_name)`	`POST /catalog/v1/types/tagdefs` *(idempotent)*
`create_kek(name, kms_type, kms_key_id, shared?)`	`POST /dek-registry/v1/keks`
`get_kek(name)`	`GET /dek-registry/v1/keks/{name}`
`create_dek(kek, subj, algo?, encrypted_key_material?)`	`POST /dek-registry/v1/keks/{name}/deks`
`get_dek(kek, subject)`	`GET /dek-registry/v1/keks/{name}/deks/{subject}`
`delete_version(subject, version)`	`DELETE /subjects/{s}/versions/{v}`
`delete_subject(subject)`	`DELETE /subjects/{s}` *(soft)*
`delete_subject_permanent(subject)`	`DELETE /subjects/{s}?permanent=true`
`test_compatibility(subject, schema)`	`POST /compatibility/subjects/{s}/versions/{v}`
`get_compatibility(subject?)`	`GET /config[/{s}]`
`set_compatibility(level, subject?)`	`PUT /config[/{s}]`
`encode(schema_id, payload)`	*(local)* packs Confluent wire format
`decode_header(data)`	*(local)* validates magic byte, extracts schema ID

1.8.2 `ProtoSchema` (`proto_schema.py`)

A `typing.Protocol` (runtime-checkable) that defines the common interface for both dynamic (`ProtoMessage`) and compiled (`CompiledProtoMessage`) Protobuf schema objects. Any object exposing these attributes and methods can be used interchangeably by the SerDes layer (`KafkaProtobufSerializer` / `KafkaProtobufDeserializer`).

Attribute / Method	Purpose
`name: str`	The Protobuf message name
`file_name: str`	The `.proto` file name
`to_schema_string()`	Return the `.proto` schema text for SR registration
`serialize(data)`	Encode a dict to Protobuf binary
`deserialize(raw)`	Decode Protobuf binary to a plain dict
`save_schema(directory)`	Write the `.proto` schema text to `directory/{file_name}`

1.8.3 `ProtoMessage` / `ProtoField` (`dynamic_protobuf_helpers.py`)

Pure-Python dataclasses that build and binary-encode proto3 schemas without `protoc` or generated stubs, using the `google.protobuf` runtime directly.

Schema building — `to_schema_string()` emits valid proto3 text; `_to_fdp()` builds an equivalent `descriptor_pb2.FileDescriptorProto` programmatically, mapping every `ProtoField` to a `FieldDescriptorProto` (scalar types via `_PROTO_SCALAR_TYPES`, message-type references via `_full_type_name()`, proto3 `optional` via synthetic oneofs ordered last per the proto spec).

Descriptor pool — `message_class()` creates a fresh `descriptor_pool.DescriptorPool()` per `ProtoMessage` instance, calls `_ensure_in_pool()` to register all imported dependencies in dependency order, then calls `message_factory.GetMessageClass()` to obtain a real `google.protobuf.Message` subclass. Using a per-instance pool prevents name conflicts between schema-evolution versions of the same message.

SerDes — `serialize(data)` calls `json_format.ParseDict(data, cls())` then `.SerializeToString()`. `deserialize(raw)` calls `cls.FromString(raw)` then `json_format.MessageToDict(..., preserving_proto_field_name=True)`. Both produce and consume real Protobuf binary — no JSON stand-in.

Schema export — `save_schema(directory)` writes the proto3 schema string to `directory/{file_name}`, creating the directory if needed. This lets you export dynamic schemas as standard `.proto` files for use with `protoc` or other language toolchains.

1.8.4 `CompiledProtoMessage` / `compile_protos` / `load_compiled_message` (`compiled_protobuf_helpers.py`)

Provides the static (protoc-compiled) counterpart to the dynamic `ProtoMessage` approach. Activated by the `--use-protoc` CLI flag.

`compile_protos(proto_dir)` — Locates the `protoc` binary on `PATH`, then compiles all `.proto` files under `proto_dir` (recursively) into `_pb2.py` stubs in `src/generated_pb2/`. Creates `__init__.py` files

in all generated subdirectories for Python import resolution. Raises `FileNotFoundError` if `protoc` is not installed.

`load_compiled_message(proto_file, message_name)` — Dynamically imports the generated `_pb2` module for a given `.proto` file, extracts the named message class, and wraps it in a `CompiledProtoMessage` dataclass.

`CompiledProtoMessage` — A dataclass wrapper around a protoc-generated message class that satisfies the `ProtoSchema` protocol. Provides the same `serialize()` / `deserialize()` / `to_schema_string()` / `save_schema()` interface as `ProtoMessage`, but uses pre-compiled stubs for better performance and compile-time type safety.

Method	Purpose
`to_schema_string()`	Returns the original `.proto` schema text (read from `schemas/`)
`serialize(data)`	`ParseDict` → `SerializeToString()` using the compiled message class
`deserialize(raw)`	`ParseFromString` → `MessageToDict` using the compiled message class
`save_schema(directory)`	Writes the `.proto` schema text to `directory/{file_name}`

> **Note:** Static mode uses the global `descriptor_pool`, so it cannot load two versions of the same message name simultaneously (unlike the dynamic per-instance pool approach). This makes it unsuitable for schema-evolution demos that register multiple versions.

1.8.5 `KafkaProtobufSerializer` (`kafka_protobuf_serdes.py`)

Mirrors the Java `KafkaProtobufSerializer`. Resolves the SR subject from the topic, message name, and `is_key` flag using the configured `subject_name_strategy`, then either auto-registers the schema or looks it up, and finally calls `sr.encode()` to wrap the payload in the Confluent wire format. Maintains a module-level `_schema_id_to_message` registry so the deserializer can resolve message classes by schema ID. When a `field_encryptor` is provided along with `metadata` and `rule_set`, tagged fields are encrypted via `FieldEncryptor.encrypt_fields()` before Protobuf serialization.

1.8.6 `KafkaProtobufDeserializer` (`kafka_protobuf_serdes.py`)

Mirrors the Java `KafkaProtobufDeserializer`. Strips the wire-format header via `sr.decode_header()`, warms the schema cache via `get_schema_by_id()`, then either delegates to `specific_type.deserialize()` or looks up the message class from the module-level `_schema_id_to_message` registry populated by the serializer (DynamicMessage equivalent). When a `field_encryptor` is provided, tagged fields are automatically decrypted after deserialization using metadata from either the SR response or an in-process `_schema_id_to_csfle` cache.

1.8.7 `FieldEncryptor` / `get_encrypted_fields()` (`field_encryption.py`)

Implements AES-256-GCM field-level encryption for Confluent CSFLE via AWS KMS.

`get_encrypted_fields(metadata, rule_set)` — Inspects schema metadata property tags (e.g. `SensitiveRecord.ssn.tags = "PII"`) and rule set domain rules of type `ENCRYPT` to return the list of field names targeted for encryption.

`FieldEncryptor` — Manages Data Encryption Keys (DEKs) per subject and encrypts/decrypts individual string field values. Uses the Confluent DEK Registry for DEK storage and AWS KMS for KEK management:

1. **First encrypt** — generates a 256-bit DEK locally, encrypts it via `kms.encrypt()`, then registers the encrypted key material with `create_dek()`
2. **Subsequent calls** — `get_dek()` returns the encrypted DEK from the registry, decrypted via `boto3 kms.decrypt()`
3. **DEKs are cached** in-process per subject to avoid repeated KMS calls

Encrypted values use the envelope format: `magic(0xC0) + version(0x01) + nonce(12 bytes) + ciphertext+tag(N bytes)`, base64-encoded before storage in the Protobuf string field.

Method	Purpose
`encrypt_value(value, subject)`	Encrypt a single string → base64 ciphertext
`decrypt_value(encrypted, subject)`	Decrypt a base64 ciphertext → original string
`encrypt_fields(data, fields, subject)`	Batch-encrypt named fields in a data dict
`decrypt_fields(data, fields, subject)`	Batch-decrypt named fields in a data dict
`_encrypt_dek_via_kms(plaintext_key)`	Encrypt a DEK via AWS KMS `kms.encrypt()`
`_decrypt_dek_via_kms(encrypted_key)`	Decrypt a DEK via AWS KMS `kms.decrypt()`

1.8.8 `kafka_helpers.py`

Contains all Kafka broker interaction logic, isolated from the demo and Schema Registry code. Only used when running with `--mode full`.

Function	Purpose
`_base_kafka_config(cfg)`	Builds the shared `bootstrap.servers` / `SASL_SSL` / `PLAIN` config dict
`ensure_topics(cfg, topics)`	`AdminClient` → `list_topics()` → `create_topics()` (rf=3, partitions=6, idempotent)
`kafka_produce(cfg, topic, key, value)`	`Producer` → `produce()` + `flush()`
`kafka_consume_one(cfg, topic, group_id)`	`Consumer` → `subscribe()` → `poll()` loop + `commit()`

1.8.9 `utilities.py`

Function	Purpose

Function	Purpose
`setup_logging(log_file?)`	Reads `[tool.logging]` from `pyproject.toml` via `tomllib` and applies it with `logging.config.dictConfig()`. Falls back to a basic dual-handler setup (file + console) if the config is absent. Returns the root logger.
`get_config()`	Reads the seven environment variables (`BOOTSTRAP_SERVERS`, `KAFKA_API_KEY`, … , `AWS_KMS_KEY_ARN`) and returns `(cfg_dict, missing_keys)`.
`parse_args()`	Defines the `--mode`, `--demo`, `--run-id`, `--save-schemas`, and `--use-protoc` CLI flags via `argparse` and returns the parsed `Namespace`.

1.8.10 `demos.py`

Contains all ten demo functions extracted from the former monolithic `main.py`. Each function receives a `SchemaRegistryClient`, an optional Kafka config dict (for `--mode full`), the `run_id` suffix, an optional `save_dir` path, and a `use_protoc` flag. When `use_protoc` is `True`, demos use `load_compiled_message()` to load protoc-compiled stubs instead of building schemas dynamically. When `save_dir` is set (via `--save-schemas`), each demo writes its `.proto` schema files to the specified directory. The module has its own logger via `setup_logging()`.

Function	Demo
`demo_basic()`	1 — Basic Serializer & Deserializer
`demo_delete_protection()`	2 — Reference-Deletion Protection
`demo_evolution()`	3 — Schema Evolution
`demo_oneof()`	4 — Multiple Event Types / `oneOf`
`demo_null_handling()`	5 — Null-Value Handling
`demo_compatibility()`	6 — Compatibility Rules
`demo_types()`	7 — Schema Types
`demo_strategies()`	8 — Subject Name Strategies
`demo_csfle()`	9 — Client-Side Field Level Encryption
`demo_no_auto_register()`	10 — Manual Schema Registration (`auto_register=False`)

1.9 Logging

Configured in `pyproject.toml` under `[tool.logging]` and loaded at startup by `utilities.setup_logging()`:

Handler	Level	Output

Handler	Level	Output
`console`	`DEBUG`	stdout
`file`	`INFO`	`cc-python-dynamic_static-protobuf-example.log` (overwritten each run, mode `w`)

Log format: `YYYY-MM-DD HH:MM:SS - LEVEL - function_name - message`

The fallback log filename and format are defined as typed `Final` constants in `constants.py` (`DEFAULT_TOOL_LOG_FILE`, `DEFAULT_TOOL_LOG_FORMAT`).

1.10 Wire format

Every serialized Kafka message uses the Confluent wire format— the envelope that wraps protobuf binary before it goes onto a Kafka topic:

```
┌──────────┬──────────────────────────┬──────────────────────┬──────────────
──────────┐
│  magic   │  schema_id               │  message index       │  payload
│
│  1 byte  │  4 bytes (big-endian)    │  varint array        │  N bytes
│
│  0x00    │  e.g. 0x00000042 = 66    │  0x00 (first msg)    │  Protobuf
binary      │
└──────────┴──────────────────────────┴──────────────────────┴──────────────
──────────┘
```

Segment	Bytes	Purpose
Magic byte	`0x00`	Identifies this as a Confluent Schema Registry message
Schema ID	4 bytes, big-endian	Points to the schema stored in Schema Registry
Message index	Variable (varint-encoded)	Tells which message type in the .proto is encoded
Payload	Remaining bytes	The actual protobuf binary

1.10.1 Why a message index?

A single `.proto` file (or schema) can define multiple message types:

```
message Customer { ... }   // index [0]
message Product { ... }   // index [1]
message Order { ... }    // index [2]
```

The message index tells the consumer which one was serialized. When there's only one message (the common case), the index is an empty array encoded as the single byte `0x00`.

1.10.2 What the code does

In `schema_registry_client.py`:

`encode()` builds the envelope:

```
struct.pack(">bI", 0x00, schema_id) + b"\x00" + payload
            magic┘     └schema ID  └message index └payload
```

Then appends the protobuf binary after it.

`decode_header()` does the reverse:

- Validates the magic byte is `0x00`
- Unpacks the 4-byte schema ID
- Skips past the message-index varint array using `_read_varint()`
- Returns the remaining bytes as the protobuf payload

This envelope is what makes Schema Registry-aware consumers (in any language) able to look up the correct schema and deserialize the message automatically.

2.0 Running the demos

```
./run-demo.sh --mode=<schema-only|full> \
              [--profile=<SSO_PROFILE_NAME>] \
              [--demo=
<all|basic|delete|evolution|oneof|null|compat|types|strategies|csfle|no-
auto-register>] \
              [--run-id=<any string, e.g. "test1">] \
              [--save-schemas=<directory>] \
              [--use-protoc]
```

Argument	Required	Choice	Default	Description
`--mode`	✅	`schema-only`, `full`	`schema-only`	SR-only or full Kafka round-trip

Argument	Required	Choice	Default	Description
`--profile`	❌	---	---	The AWS SSO profile name. Passed directly to `aws sso login` and `aws2-wrap` for authentication, and used to resolve `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`, which are then exported as `TF_VAR_aws_region`, `TF_VAR_aws_access_key_id`, `TF_VAR_aws_secret_access_key`, and `TF_VAR_aws_session_token` for Terraform, respectively.
`--demo`	❌	`all` `basic` `delete` `evolution` `oneof` `null` `compat` `types` `strategies` `csfle` `no-auto-register`	`all`	Which demo to run
`--run-id`	❌	any string	random 8-char UUID prefix	Appended to every topic and subject name to prevent collisions across runs
`--save-schemas`	❌	directory path	disabled	Save generated `.proto` schema files to the given directory (created if needed)
`--use-protoc`	❌	flag (no value)	disabled	Use protoc-compiled `_pb2.py` stubs instead of dynamic runtime descriptors. Requires `protoc` on `PATH` (`brew install protobuf`).

> All required flag(s) must be provided; if a required flag is missing, the script exits with code `85`.

In `--mode=full`, the app calls `ensure_topics()` via `AdminClient` to pre-create all five required topics before any produce calls. Confluent Cloud mandates `replication_factor=3`; existing topics are silently skipped. Schema registration and Kafka produce/consume are fully integrated in both modes, but only in `full` mode do the messages actually go to Kafka. In `schema-only` mode, the app still registers schemas and prints the resulting wire-format bytes to the console, but does not interact with Kafka at all.

2.1 Demos

2.1.1 Demo 1 — Basic Serializer & Deserializer (`--demo basic`)

Builds two `ProtoMessage` objects (`OtherRecord` and `MyRecord`), registers them in dependency order (referenced schema first), and serializes a message into the Confluent wire format. Prints the magic byte (`0x00`), schema ID, and full hex payload. Deserializes back with `KafkaProtobufDeserializer` using `specific_type=my_record`. In `full` mode the wire bytes are produced to and consumed from Kafka.

Topics: `testproto-{run_id}`
Subjects: `other-{run_id}.proto`, `testproto-{run_id}-value`

2.1.2 Demo 2 — Reference-Deletion Protection (`--demo delete`)

Demonstrates that Schema Registry rejects deletion of a schema that is referenced by another. Calls `referenced_by()` to show the dependency graph, attempts to delete the leaf subject (expects a `RuntimeError`), then shows the correct order: delete the referencing subject first, then the referenced one.

2.1.3 Demo 3 — Schema Evolution (`--demo evolution`)

Registers a v1 `MyRecord` schema (`id`, `amount`), sets `BACKWARD_TRANSITIVE` compatibility on the subject via `PUT /config/{subject}`, then registers v2 with an added `customer_id` field. Calls `test_compatibility()` before registration to verify safety. In `full` mode, produces both schema versions to the same topic and consumes them.

Topics: `transactions-proto-{run_id}`
Subjects: `transactions-proto-{run_id}-value`

2.1.4 Demo 4 — Multiple Event Types / `oneOf` (`--demo oneof`)

Builds four schemas: `Customer`, `Product`, `Order`, and an `AllTypes` wrapper that holds all three under a `oneof oneof_type` field. Registers all four with cross-schema references, then serializes and routes three heterogeneous events (`customer`, `product`, `order`) through a single topic.

Topics: `all-events-{run_id}`
Subjects: `Customer-{run_id}.proto`, `Product-{run_id}.proto`, `Order-{run_id}.proto`, `all-events-{run_id}-value`

2.1.5 Demo 5 — Null-Value Handling (`--demo null`)

Shows the recommended proto3 approach for nullable fields using the `optional` keyword on `ProtoField`. Serializes a partial record (name only) and a full record, then deserializes both. Also prints the pre-proto3 `google.protobuf.StringValue` wrapper alternative used with Confluent Connect's `wrapper.for.nullable=true` flag.

Topics: `nullables-{run_id}`

2.1.6 Demo 6 — Compatibility Rules (`--demo compat`)

Fetches the SR global compatibility level via `GET /config` and prints a reference table of all seven modes. Highlights the key Protobuf-vs-Avro distinction: adding a new *message type* (not just a field) breaks FORWARD compatibility, making `BACKWARD_TRANSITIVE` the recommended default.

2.1.7 Demo 7 — Schema Types & Return Types (`--demo types`)

Calls `GET /schemas/types` to list which schema types your SR instance supports, then prints a reference table mapping Avro / Protobuf / JSON Schema to their specific and generic Python return types. Documents the Python equivalents of the Java `specific.protobuf.value.type` and `derive.type` config properties.

2.1.8 Demo 8 — Subject Name Strategies (`--demo strategies`)

Registers the same `Payment` schema under all three subject name strategies and prints the resulting subject name for each. Also documents both reference subject name strategies.

Strategy	Resulting subject
`TopicNameStrategy`	`payments-{run_id}-value`
`RecordNameStrategy`	`Payment`
`TopicRecordNameStrategy`	`payments-{run_id}-Payment`
`DefaultReferenceSubjectNameStrategy`	import path, e.g. `other.proto`
`QualifiedReferenceSubjectNameStrategy`	dotted form, e.g. `mypackage.myfile`

2.1.9 Demo 9 — Client-Side Field Level Encryption (`--demo csfle`)

Demonstrates Confluent-native CSFLE using the **`confluent-kafka`** library's `ProtobufSerializer` / `ProtobufDeserializer` with the `FieldEncryptionExecutor` rule engine and `AwsKmsDriver`. This is the fully integrated Confluent approach — encryption/decryption is handled transparently by the serializer/deserializer rather than by the custom `FieldEncryptor` used in other demos.

The demo proceeds in eleven steps:

1. **Register drivers** — `AwsKmsDriver.register()` + `FieldEncryptionExecutor.register()`
2. **Build schema** — a `SensitiveRecord` with `ssn` and `email` fields tagged `PII` via `ProtoField(tags="PII")`
3. **Define rule set** — an `ENCRYPT / WRITEREAD` domain rule matching `PII` tags, pointing to the KEK and AWS KMS key ID
4. **Register tag + KEK** — `sr.create_tag("PII")` (idempotent) then `sr.create_kek()` to register the Key Encryption Key in the DEK Registry
5. **Register schema with rules** — `sr.register(subject, schema, rule_set=rule_set)` embeds the encryption rule into the SR subject
6. **Instantiate Confluent SR client** — a second `SchemaRegistryClient` from `confluent_kafka.schema_registry` (required by the Confluent SerDes)
7. **Build SerDes** — `ProtobufSerializer(msg_cls, confluent_sr, {'auto.register.schemas': False, 'use.latest.version': True})` and matching `ProtobufDeserializer`
8. **Serialize** — `protobuf_serializer(msg_instance, SerializationContext(...))` — `FieldEncryptionExecutor` automatically fetches the DEK from the DEK Registry (creating it via

KMS on first call), encrypts `ssn` and `email` with AES-256-GCM, and emits Confluent wire-format bytes
9. **Deserialize** — `protobuf_deserializer(wire, SerializationContext(...))` — `FieldEncryptionExecutor` fetches and decrypts the DEK, restores plaintext
10. **Kafka round-trip** — produce/consume the wire bytes in `--mode full`
11. **Architecture notes** — printed to log showing the full CSFLE flow

Requires `AWS_KMS_KEY_ARN` and valid AWS credentials (`boto3`-compatible: env vars, `~/.aws/credentials`, IAM role, or AWS SSO via `run-demo.sh`).

Topics: `csfle-{run_id}` **Subjects:** `csfle-{run_id}-value`

2.1.10 Demo 10 — Manual Schema Registration (`--demo no-auto-register`)

Demonstrates the `auto_register=False` serializer mode, where schemas must be pre-registered before any produce calls. The demo walks through three steps:

1. **Step 1** — Manually registers an `Invoice` schema under `invoices-{run_id}-value` via `sr.register()` and prints the returned `schema_id`.
2. **Step 2** — Creates a `KafkaProtobufSerializer(sr, auto_register=False)`, serializes an invoice record, and performs a round-trip deserialize (with an optional Kafka produce/consume in `--mode full`).
3. **Step 3** — Shows the expected `RuntimeError` when the same serializer tries to produce to an *unregistered* subject (`unknown-{run_id}`), proving that `auto_register=False` enforces strict governance.

The demo closes with a reference box explaining when to use `auto_register=False` in production (CI/CD-managed schemas, read-only producer credentials, strict schema governance).

Topics: `invoices-{run_id}` **Subjects:** `invoices-{run_id}-value`

3.0 Cleanup

Since all topics and subjects are suffixed with `<run_id>`, to remove everything a specific run created, follow these steps:

```
# Soft-delete SR subjects
confluent schema-registry subject list \
  | grep '<run_id>' \
  | xargs -I{} confluent schema-registry subject delete --subject {} --
force

# Delete Kafka topics
for topic in testproto transactions-proto all-events nullables payments
csfle invoices; do
  confluent kafka topic delete "${topic}-<run_id>"
done
```

4.0 Notes

- **No standalone SR Python library.** There is no `confluent-schema-registry` PyPI package. `SchemaRegistryClient` calls the REST API directly via `requests` — this is the correct approach for Python as recommended by Confluent. Besides, the code caches schemas by ID in a dict to avoid redundant `GET /schemas/ids/{id}` calls. Moreover, since the code is already caching schemas by ID, HTTP calls are only made on cache misses, so the performance difference between a hand-rolled client and a hypothetical official library would be negligible.
- **Confluent Cloud requires `replication_factor=3`.** `ensure_topics()` always passes this value; any other value will be rejected by the cluster.
- **`BACKWARD_TRANSITIVE` is the right default for Protobuf.** Unlike Avro, adding a new *message type* (not just a field) breaks `FORWARD` compatibility.
- **`uv.lock` should be committed.** It pins every transitive dependency for fully reproducible installs across machines and CI.
- **Real Protobuf binary encoding.** Both `ProtoMessage` (dynamic) and `CompiledProtoMessage` (static) produce real Protobuf binary on the wire. The dynamic path uses `google.protobuf.message_factory` with runtime-constructed `FileDescriptorProto` objects — no `protoc` required. The static path (`--use-protoc`) compiles `.proto` files from `schemas/` into `_pb2.py` stubs via `protoc` for better performance and compile-time type safety. Both satisfy the `ProtoSchema` protocol and are interchangeable in the SerDes layer. `json_format.ParseDict` / `MessageToDict` bridge between plain Python dicts and `google.protobuf.Message` instances in both modes.

5.0 Resources

- Confluent Protobuf SerDes documentation
- Protocol Buffers (or Protobuf for short) documentation
- Protect Sensitive Data Using Client-Side Field Level Encryption on Confluent Cloud
- Confluent Kafka Python Protobuf Producer Encryption Example